

When the Attack Is Not an Attack: Validity Failures in LLM-Generated Red-Teaming

Yihang Jiao · yihangj3@illinois.edu · <https://github.com/yhj3/counterfactual-textattack>

Automated red-teaming uses one language model to generate the probes that test another. A probe only tells us something about the target if it is a valid test of the target. I measure four ways it fails to be, on 32 configurations and 600 fresh generations.

Label validity — overstates by $3.05\times$

A task-specific rewriting prompt roughly doubles the attacks that survive (**119/180** vs. 58/180, $p = 1.7 \times 10^{-10}$, dataset and target held fixed). On sentiment this is free (similarity 0.713 vs. 0.705). On topic classification it is not — similarity collapses **0.691** $\rightarrow$ **0.361**, because that prompt instructs the model to change the topic “from world to sports”. A text whose topic really changed is a different input with a different correct label, so a prediction change on it is the classifier behaving *correctly*.

	Flip rate	Meaning-preserving	Ratio
AG News	65.0%	3.3%	$19.5\times$
SST-2	50.0%	15.0%	$3.3\times$
IMDb	83.3%	46.7%	$1.8\times$
Pooled	66.1%	21.7%	$3.05\times$

The instruction most effective at producing label flips is the instruction to change the label. Any prompt search that optimizes reported attack success will find it.

Input validity — understates, so the errors do not cancel

Sentence-pair tasks need two segments. Most probes merge them and are not well-formed inputs at all (100 of 198 in the stored corpus; for RTE under PWWS, **0 of 31**). Malformed probes never register a flip, so leaving them in the denominator counts them as attack failures:

Prompt	Well-formed	Unchecked	Valid only
generic	0.300	12.5%	41.7%
structured	0.575	21.7%	37.7%

Stating the format constraint nearly doubles validity ($p = 2.8 \times 10^{-5}$), but the model still violates an explicit format over 40% of the time. Label validity inflates the reported number, input validity deflates it — **the two errors push in opposite directions and do not cancel.**

Generator validity — the red-teamer itself refuses

The generator is safety-trained and sometimes declines: “*I cannot provide a revised text that could potentially be used to mislead or deceive...*” Of 600 generations, **37 (6.2%)** begin with a refusal, and **7 of those are scored as successful attacks** (mean similarity 0.216 to the original). It is not evenly spread over the conditions being compared: **11.0%** under the generic prompt against **0.6%** under the task-specific one, and **17.5%** on RTE. Excluding refusals leaves the results above intact (118/179 vs. 52/170, $p = 4.2 \times 10^{-11}$), but a refusal and a failed attack look identical in the output column, so the check is not optional.

What the filter trades

Per-candidate filtering on all three conditions, 360 probes:

Stage	Kept	Frac.	Mean sim. / median ppl
all probes	360	1.000	0.648 / 33.8
+ flips the classifier	177	0.492	0.596 / 28.1
+ similarity ≥ 0.75	64	0.178	0.840 / 27.2
+ perplexity ≤ 150	55	0.153	0.834 / 22.9

Retains **15.3%**, and **similarity does the filtering, not fluency**: it removes 113 of the 177 that flip, perplexity 9 more. Fluency is nearly free — TextFooler’s own perturbations sit at median perplexity **522** against 22.9. Underneath everything, rewriting keeps only **44.5%** of the attacks it is applied to: a trade, not an improvement.

Four numbers a red-teaming report should carry. (1) semantic similarity beside every flip rate — on AG News, the difference between 65.0% and 3.3%; (2) the fraction of generations that are well-formed inputs — on pair tasks, 12.5% vs. 41.7%; (3) the generator’s refusal rate per condition — here 11.0% vs. 0.6%; (4) the number of generations actually scored by the target model — without it a pipeline can report a metric computed from something other than what it generated, and nothing will look wrong.